Introduction to Statistical Learning with Applications

CM10: Tabular Foundational Models

Pedro L. C. Rodrigues

- Introduction
- From point prediction to distributions
- Gaussian processes
- Conditional neural processes
- Tabular foundational models

Introduction

Today we will talk about a completely new paradigm in machine learning

Tabular Foundational Models

Introduction

Today we will talk about a completely new paradigm in machine learning

Tabular Foundational Models

But first of all… what are tabular data?

$$\begin{bmatrix} \mathbf{X} \\ (n \times p) \end{bmatrix} \begin{bmatrix} \mathbf{y} \\ (n \times p) \end{bmatrix}$$

It is the most common type of data that you will see when working as data scientist…

Introduction

Today we will talk about a completely new paradigm in machine learning

Tabular Foundational Models

But first of all… what are tabular data?

The titanic dataset

	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	eml	boat	body	home.dest
0	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S	2		St Louis, MO
1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.55	C22 C26	S	11		Montreal, PQ / Chesterville, ON
2	1	Allison, Miss. Helen Loraine	female	2.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
3	1	Allison, Mr. Hudson Joshua Creighton	male	30.0	1	2	113781	151.55	C22 C26	S		135.0	Montreal, PQ / Chesterville, ON
4	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
5	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.55	E12	S	3		New York, NY
6	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S	10		Hudson, NY
7	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S			Belfast, NI
8	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S	D		Bayside, Queens, NY
9	1	Artagaveytia, Mr. Ramon	male	71.0	0	0	PC 17609	49.5042		C		22.0	Montevideo, Uruguay
10	1	Astor, Col. John Jacob	male	47.0	1	0	PC 17757	227.525	C62 C64	C		124.0	New York, NY

It is the most common type of data that you will see when working as data scientist…

Introduction

Today we will talk about a completely new paradigm in machine learning

Tabular Foundational Models

But first of all… what are tabular data?

```
fit(X, y, sample_weight=None) method of sklearn.linear_model._base.LinearRegression
    Fit linear model.

    Parameters
    ----------
    X : {array-like, sparse matrix} of shape (n_samples, n_features)
        Training data.

    y : array-like of shape (n_samples,) or (n_samples, n_targets)
        Target values. Will be cast to X's dtype if necessary.

    sample_weight : array-like of shape (n_samples,), default=None
        Individual weights for each sample.
```

It is the most common type of data that you will see when working as data scientist…

Introduction

Neural networks used to **struggle** with tabular data because of…

- Heterogeneity between datasets

	EDUCATION	SOUTH	SEX	EXPERIENCE	UNION	AGE	RACE	OCCUPATION	SECTOR	MARR
0	8	no	female	21	not_member	35	Hispanic	Other	Manufacturing	Married
1	9	no	female	42	not_member	57	White	Other	Manufacturing	Married
2	12	no	male	1	not_member	19	White	Other	Manufacturing	Unmarried
3	12	no	male	4	not_member	22	White	Other	Other	Unmarried
4	12	no	male	17	not_member	35	White	Other	Other	Married
5	13	no								
6	10	yes								
7	12	no								
8	16	no								
9	12	no								
10	12	no								
11	12	no								

	mcg	gvh	alm	mit	erl	pox	vac	nuc
0	0.58	0.61	0.47	0.13	0.5	0.0	0.48	0.22
1	0.43	0.67	0.48	0.27	0.5	0.0	0.53	0.22
2	0.64	0.62	0.49	0.15	0.5	0.0	0.53	0.22
3	0.58	0.44	0.57	0.13	0.5	0.0	0.54	0.22
4	0.42	0.44	0.48	0.54	0.5	0.0	0.48	0.22
5	0.51	0.4	0.56	0.17	0.5	0.5	0.49	0.22

	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	eml	boat	body	home.dest
0	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S	2		St Louis, MO
1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.55	C22 C26	S	11		Montreal, PQ / Chesterville, ON
2	1	Allison, Miss. Helen Loraine	female	2.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
3	1	Allison, Mr. Hudson Joshua Creighton	male	30.0	1	2	113781	151.55	C22 C26	S		135.0	Montreal, PQ / Chesterville, ON
4	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
5	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.55	E12	S	3		New York, NY
6	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S	10		Hudson, NY
7	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S			Belfast, NI
8	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S	D		Bayside, Queens, NY
9	1	Artagaveytia, Mr. Ramon	male	71.0	0	0	PC 17609	49.5042		C		22.0	Montevideo, Uruguay
10	1	Astor, Col. John Jacob	male	47.0	1	0	PC 17757	227.525	C62 C64	C		124.0	New York, NY

Introduction

Neural networks used to **struggle** with tabular data because of…

- Heterogeneity between columns within a dataset

	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	eml	boat	body	home.dest
0	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S	2		St Louis, MO
1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.55	C22 C26	S	11		Montreal, PQ / Chesterville, ON
2	1	Allison, Miss. Helen Loraine	female	2.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
3	1	Allison, Mr. Hudson Joshua Creighton	male	30.0	1	2	113781	151.55	C22 C26	S		135.0	Montreal, PQ / Chesterville, ON
4	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
5	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.55	E12	S	3		New York, NY
6	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S	10		Hudson, NY
7	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S			Belfast, NI
8	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S	D		Bayside, Queens, NY
9	1	Artagaveytia, Mr. Ramon	male	71.0	0	0	PC 17609	49.5042		C		22.0	Montevideo, Uruguay
10	1	Astor, Col. John Jacob	male	47.0	1	0	PC 17757	227.525	C62 C64	C		124.0	New York, NY

features with different scales, with different types : boolean, categorical, ordinal, integer, floating point; imbalance or missing data; unimportant features; outliers

Introduction

Neural networks used to **struggle** with tabular data because of…

- Column names actually convey information!

	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	eml	boat	body	home.dest
0	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S	2		St Louis, MO
1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.55	C22 C26	S	11		Montreal, PQ / Chesterville, ON
2	1	Allison, Miss. Helen Loraine	female	2.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
3	1	Allison, Mr. Hudson Joshua Creighton	male	30.0	1	2	113781	151.55	C22 C26	S		135.0	Montreal, PQ / Chesterville, ON
4	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0	1	2	113781	151.55	C22 C26	S			Montreal, PQ / Chesterville, ON
5	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.55	E12	S	3		New York, NY
6	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S	10		Hudson, NY
7	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S			Belfast, NI
8	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S	D		Bayside, Queens, NY
9	1	Artagaveytia, Mr. Ramon	male	71.0	0	0	PC 17609	49.5042		C		22.0	Montevideo, Uruguay
10	1	Astor, Col. John Jacob	male	47.0	1	0	PC 17757	227.525	C62 C64	C		124.0	New York, NY

Introduction

Neural networks used to **struggle** with tabular data because of…

Nevertheless, research kept going 💪

Because it would be cool to:

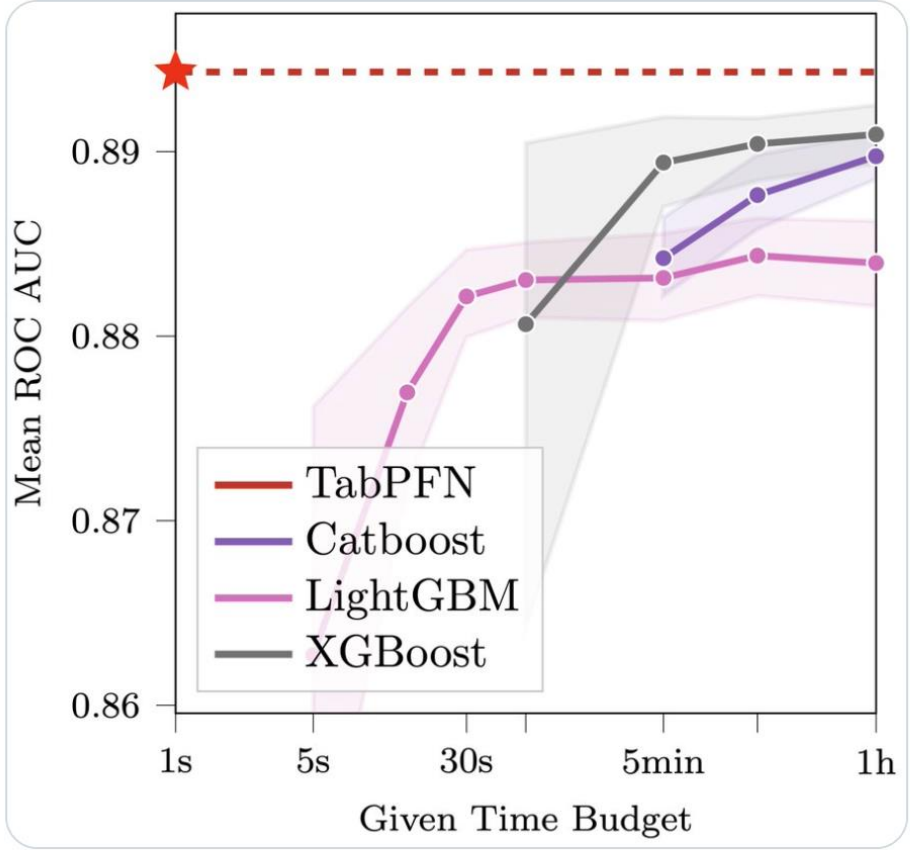
- Leverage modern deep learning infrastructure (e.g. GPUs)
- Have a differentiable pipeline for end-to-end learning
- Multimodal processing with text and images

And in 2022…

Introduction

Frank Hutter @FrankRHutter · Oct 21, 2022

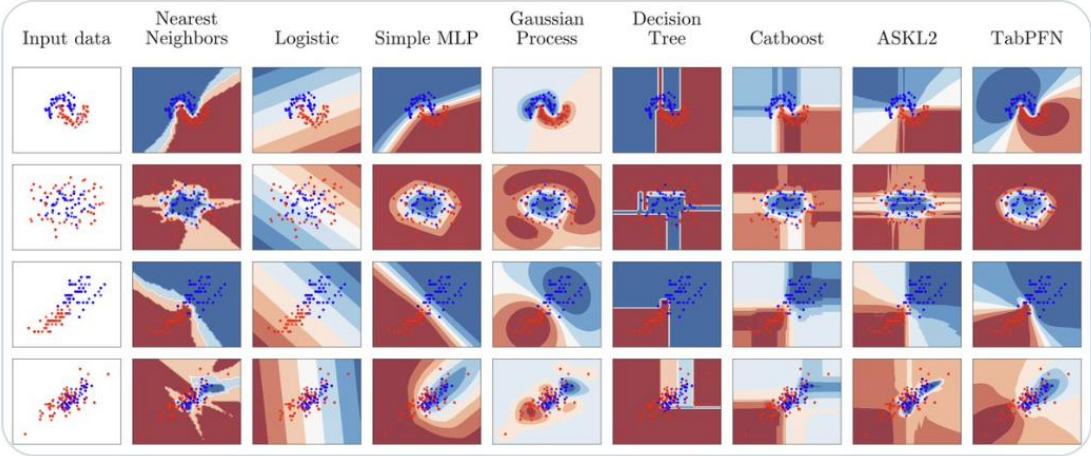
This may revolutionize data science: we introduce TabPFN, a new tabular data classification method that takes 1 second & yields SOTA performance (better than hyperparameter-optimized gradient boosting in 1h). Current limits: up to 1k data points, 100 features, 10 classes. 🧵1/6



104 867 4.1K

Frank Hutter @FrankRHutter · Oct 21, 2022

TabPFN is radically different from previous ML methods. It is meta-learned to approximate Bayesian inference with a prior based on principles of causality and simplicity. Here's a qualitative comparison to some sklearn classifiers, showing very smooth uncertainty estimates. 2/6



5 26 263

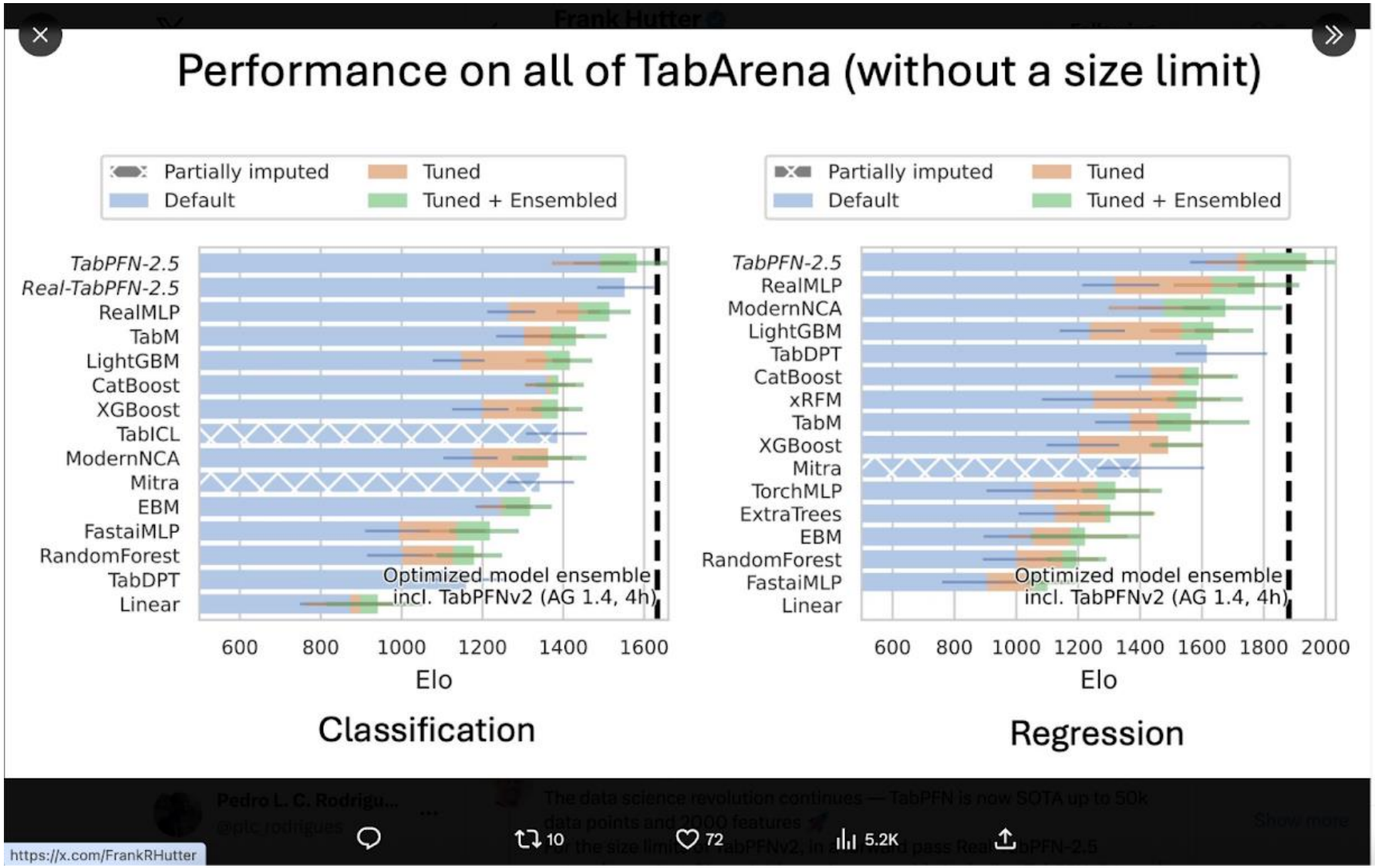
Frank Hutter @FrankRHutter · Oct 21, 2022

TabPFN happens to be a transformer, but this is not your usual trees vs nets battle. Given a new data set, there is no costly gradient-based training. Rather, it's a single forward pass of a fixed network: you feed in (Xtrain, ytrain, Xtest); the network outputs p(y_test). 3/6

3 9 122

https://x.com/FrankRHutter/status/1583410845307977733

Introduction



Frank Hutter @FrankRHutter

Update: TabPFN-2.5 is actually the SOTA model on all of TabArena (which has datasets with up to 100k training data points).
In a single forward pass, TabPFN-2.5 outperforms all other models, even if you tune them for 4 hours.

We built and previously evaluated TabPFN-2.5 for up to 50k data points (and 2k features) and were kind of surprised that it's SOTA up to 100k 🙂

👉 TabPFN-2.5 webinar tomorrow: app.livestorm.co/p/21526e44-406...
👉 Model report on arXiv: arxiv.org/pdf/2511.08667

10:40 AM · Nov 13, 2025 · **5,239** Views

10 72 35

Post your reply Reply

Introduction

Today we will talk about a completely new paradigm in machine learning

Tabular Foundational Models

What is a foundational model?

- Large-scale models (e.g., BERT, DALL-E, GPT-3) trained on broad data that can be adapted to a wide range of downstream tasks

And a tabular foundational model (TFM)?

- A model trained on broad tabular data that can be adapted to a wide range of downstream tasks

Introduction

In practice, this means that for a fixed pre-trained neural network g_θ we can handle datasets of different sizes and natures

	EDUCATION	SOUTH	SEX	EXPERIENCE	UNION	AGE	RACE	OCCUPATION	SECTOR	MARR
0	8	no	female	21	not_member	35	Hispanic	Other	Manufacturing	Married
1	9	no	female	42	not_member	57	White	Other	Manufacturing	Married
2	12	no	male	1	not_member	19	White	Other	Manufacturing	Unmarried
3	12	no	male	4	not_member	22	White	Other	Other	Unmarried
4	12	no	male	17	not_member	35	White	Other	Other	Married
5	13	no	male	9	member	28	White	Other	Other	Unmarried
6	10	yes	male	27	not_member	43	White	Other	Other	Unmarried
7	12	no	male	9	not_member	27	White	Other	Other	Unmarried
8	16	no	male	11	not_member	33	White	Other	Manufacturing	Married
9	12	no	male	9	not_member	27	White	Other	Other	Unmarried
10	12	no	male	17	member	35	White	Other	Other	Married
11	12	no	male	19	member	37	White	Other	Manufacturing	Unmarried

$$X_1 \in \mathbb{R}^{n_1 \times p_1}$$
$$y_1 \in \mathbb{R}^{n_1}$$

	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	eml	boat
0	1	Allen, Miss. Elisabeth Walton	female	29.0	0	0	24160	211.3375	B5	S	2
1	1	Allison, Master. Hudson Trevor	male	0.9167	1	2	113781	151.55	C22 C26	S	11
2	1	Allison, Miss. Helen Loraine	female	2.0	1	2	113781	151.55	C22 C26	S	
3	1	Allison, Mr. Hudson Joshua Creighton	male	30.0	1	2	113781	151.55	C22 C26	S	
4	1	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0	1	2	113781	151.55	C22 C26	S	
5	1	Anderson, Mr. Harry	male	48.0	0	0	19952	26.55	E12	S	3
6	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9583	D7	S	10
7	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S	
8	1	Appleton, Mrs. Edward Dale (Charlotte Lamson)	female	53.0	2	0	11769	51.4792	C101	S	D
9	1	Artagaveytia, Mr. Ramon	male	71.0	0	0	PC 17609	49.5042		C	
10	1	Astor, Col. John Jacob	male	47.0	1	0	PC 17757	227.525	C62 C64	C	

$$X_2 \in \mathbb{R}^{n_2 \times p_2}$$
$$y_2 \in \{0,1\}^{n_2}$$

$$\boxed{y_\star = g_\theta(x_\star, \underbrace{X_i, y_i}_{\text{context}})}$$

	mcg	gvh	alm	mit	erl	pox	vac	nuc
0	0.58	0.61	0.47	0.13	0.5	0.0	0.48	0.22
1	0.43	0.67	0.48	0.27	0.5	0.0	0.53	0.22
2	0.64	0.62	0.49	0.15	0.5	0.0	0.53	0.22
3	0.58	0.44	0.57	0.13	0.5	0.0	0.54	0.22
4	0.42	0.44	0.48	0.54	0.5	0.0	0.48	0.22
5	0.51	0.4	0.56	0.17	0.5	0.5	0.49	0.22
6	0.5	0.54	0.48	0.65	0.5	0.0	0.53	0.22
7	0.48	0.45	0.59	0.2	0.5	0.0	0.58	0.34
8	0.55	0.5	0.66	0.36	0.5	0.0	0.49	0.22
9	0.4	0.39	0.6	0.15	0.5	0.0	0.58	0.3
10	0.43	0.39	0.54	0.21	0.5	0.0	0.53	0.27
11	0.42	0.37	0.59	0.2	0.5	0.0	0.52	0.29

$$X_3 \in \mathbb{R}^{n_3 \times p_3}$$
$$y_3 \in \{0,1,2\}^{n_3}$$

The model learns to adapt at inference time! But how?

Introduction

Key concept: TFMs leverage **in-context learning** (ICL)

For a given dataset $(X, y) = (X_{\text{train}}, y_{\text{train}}) \cup (X_{\text{test}}, y_{\text{test}})$

Supervised learning

Train f_θ with $(X_{\text{train}}, y_{\text{train}})$

Predict on $\hat{y}_{\text{test}} = f_\theta(X_{\text{test}})$

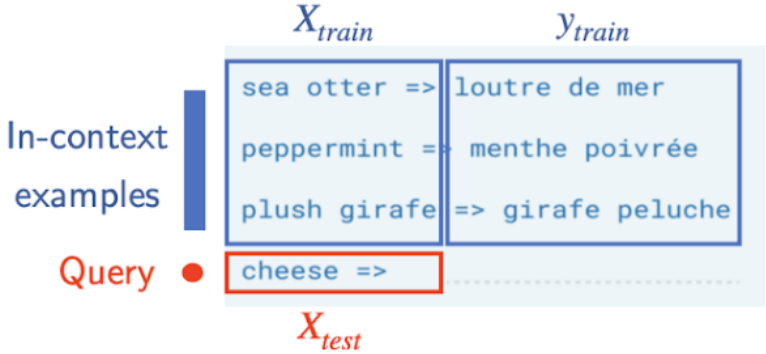
In-context learning

Directly predict

$$\hat{y}_{\text{test}} = g_\theta(X_{\text{test}}; X_{\text{train}}, y_{\text{train}})$$

estimator pre-trained on several datasets

context



ICL is an **emergent** property that appears in very large transformer-based neural networks when training of very big datasets. It allows a model to performs tasks by analyzing prompts containing input-output pairs, without explicit training or parameter updates

Introduction

🔑 Key concept: TFMs leverage **in-context learning** (ICL)

For a given dataset $(X, y) = (X_{\text{train}}, y_{\text{train}}) \cup (X_{\text{test}}, y_{\text{test}})$

Supervised learning

Train f_θ with $(X_{\text{train}}, y_{\text{train}})$

Predict on $\hat{y}_{\text{test}} = f_\theta(X_{\text{test}})$

In-context learning

Directly predict

$$\hat{y}_{\text{test}} = \underbrace{g_\theta}_{\text{estimator pre-trained on several datasets}}(X_{\text{test}}; \underbrace{X_{\text{train}}, y_{\text{train}}}_{\text{context}})$$

💡 The cost of training this very large model g_θ is amortized

We will see later how we can **"teach" this ICL feature** to the network…

Introduction

https://github.com/soda-inria/tabicl

```
pip install tabicl
```

```
from tabicl import TabICLClassifier

clf = TabICLClassifier()
clf.fit(X_train, y_train)  # this is cheap
clf.predict(X_test)  # in-context learning happens here
```

But first, let's play a little bit...

Introduction

Roadmap of today's lecture…

ordinary linear regression

$$y_\star = \beta^T x_\star$$

1

Introduction

Roadmap of today's lecture...

ordinary linear regression

$$y_\star = \beta^T x_\star$$

1

impose a prior on β

to get uncert. quantif.

Bayesian linear regression

$$p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}})$$

2

defining the context as

$$\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$$

Introduction

Roadmap of today's lecture...

ordinary linear regression

$$y_\star = \beta^T x_\star$$

1

impose a prior on β

to get uncert. quantif.

Bayesian linear regression

$$p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}})$$

2

defining the context as

$$\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$$

we have a function turning dataset into parameters

posterior predictive distribution (PPD)

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}(y_\star; x_\star^T \mu_{\text{post}}, x_\star^T \Sigma_{\text{post}} x_\star + \sigma^2)$$

3

Introduction

Roadmap of today's lecture...

ordinary linear regression

$$y_\star = \beta^T x_\star$$

1

impose a prior on β

to get uncert. quantif.

Bayesian linear regression

$$p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}})$$

2

defining the context as

$$\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$$

we have a function turning dataset into parameters

posterior predictive distribution (PPD)

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}(y_\star; x_\star^T \mu_{\text{post}}, x_\star^T \Sigma_{\text{post}} x_\star + \sigma^2)$$

3

considering a richer feature representation $\phi(x_\star) \in \mathbb{R}^m$ we have $y_\star = \beta^T \phi(x_\star)$ and a similar expression for PPD

4

Introduction

Roadmap of today's lecture...

ordinary linear regression

$$y_\star = \beta^T x_\star \quad (1)$$

impose a prior on β

to get uncert. quantif.

Bayesian linear regression

$$p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}}) \quad (2)$$

defining the context as

$$\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$$

we have a function turning dataset into parameters

posterior predictive distribution (PPD)

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}(y_\star; x_\star^T \mu_{\text{post}}, x_\star^T \Sigma_{\text{post}} x_\star + \sigma^2) \quad (3)$$

considering a richer feature representation $\phi(x_\star) \in \mathbb{R}^m$ we have $y_\star = \beta^T \phi(x_\star)$ (4) and a similar expression for PPD

when $m \to \infty$ we can use the kernel trick to write $\phi(x_i)^T \phi(x_j) = k(x_i, x_j)$ and still get a closed-form PPD as per

gaussian processes

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}\left(y_\star; K_{x_\star}^T (K + \sigma^2 I)^{-1} y_{\text{train}}, k(x_\star, x_\star) - K_{x_\star}(K + \sigma^2 I)^{-1} K_{x_\star} + \sigma^2\right) \quad (5)$$

which is what we call a Gaussian process

Introduction

Roadmap of today's lecture...

ordinary linear regression

$$y_\star = \beta^T x_\star \quad (1)$$

impose a prior on β → to get uncert. quantif.

Bayesian linear regression

$$p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}}) \quad (2)$$

defining the context as

$$\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$$

we have a function turning dataset into parameters

posterior predictive distribution (PPD)

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}(y_\star; x_\star^T \mu_{\text{post}}, x_\star^T \Sigma_{\text{post}} x_\star + \sigma^2) \quad (3)$$

considering a richer feature representation $\phi(x_\star) \in \mathbb{R}^m$ we have $y_\star = \beta^T \phi(x_\star)$ (4) and a similar expression for PPD

when $m \to \infty$ we can use the kernel trick to write $\phi(x_i)^T \phi(x_j) = k(x_i, x_j)$ and still get a closed-form PPD as per

gaussian processes

$$p(y_\star|x_\star, \mathcal{C}) = \mathcal{N}\Big(y_\star; K_{x_\star}^T (K + \sigma^2 I)^{-1} y_{\text{train}}, k(x_\star, x_\star) - K_{x_\star}(K + \sigma^2 I)^{-1} K_{x_\star} + \sigma^2\Big) \quad (5)$$

which is what we call a Gaussian process; however, inverting n-by-n matrices is costly and choosing a kernel annoying, so we use neural networks

conditional neural processes

$$p_\theta(y_\star|x_\star, \mathcal{C}) = \mathcal{N}(y_\star; \mu_\star, \sigma_\star^2) \quad r = \frac{1}{n}\sum_{i=1}^{n} h_{\underline{\theta_2}}(x_i, y_i) \quad (\mu_\star, \sigma_\star) = g_{\underline{\theta_1}}(x_\star, r) \quad (x_i, y_i) \in \mathcal{C} \quad (6)$$

trainable parameters

Introduction

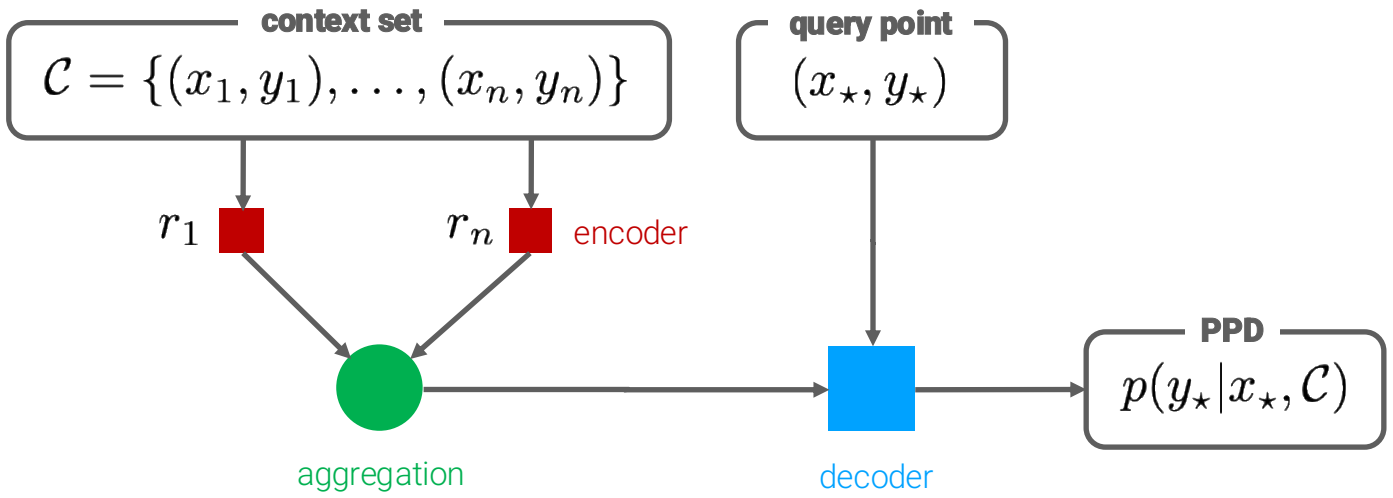
Roadmap of today's lecture...

Inverting n-by-n matrices is costly and choosing a kernel annoying, so we use neural networks

conditional neural processes

$$p_\theta(y_\star|x_\star,\mathcal{C}) = \mathcal{N}(y_\star;\mu_\star,\sigma_\star^2) \quad r = \frac{1}{n}\sum_{i=1}^{n} h_{\underline{\theta_2}}(x_i,y_i) \quad (\mu_\star,\sigma_\star) = g_{\underline{\theta_1}}(x_\star,r) \quad (x_i,y_i)\in\mathcal{C}$$

trainable parameters



Tabular foundational models use a very similar architecture, but with three important differences:

- The aggregator uses an attention mechanism instead of a simple average
- The decoder uses a softmax operator so to include classification problems on top of regression
- The training relies on a carefully designed generator of synthetic tabular datasets that go beyond smooth functions as in gaussian processes

- Introduction
- From point prediction to distributions
- Gaussian processes
- Conditional neural processes
- Tabular foundational models

From point prediction to distributions

Let us start from the very beginning : ordinary multiple linear regression ❤️

- You receive a training dataset $X_{\text{train}} \in \mathbb{R}^{n \times p}$ and $y_{\text{train}} \in \mathbb{R}^n$
- The OLS estimator is $\hat{\beta} = (X_{\text{train}}^T X_{\text{train}})^{-1} X_{\text{train}}^T y_{\text{train}}$
- Calling `LinearRegression().fit(X, y).predict(x_star)`

  returns $\hat{y}_\star = x_\star^T \hat{\beta}$ which is a **point prediction** w/o uncertainty quantification

However, we would like to work with the posterior predictive distribution (PPD)

$$p(y_\star | x_\star, X_{\text{train}}, y_{\text{train}})$$

Ok, but why? 🤷🏻‍♀️

From point prediction to distributions

We would like to work with the posterior predictive distribution (PPD)

$$p(y_\star | x_\star, X_{\text{train}}, y_{\text{train}})$$

Because

- **Full description.** A point estimate discards all uncertainty. The PPD encodes the full distribution of plausible outcomes — any summary (mean, quantiles, confidence bands) can be recovered from it.

- **Uncertainty quantification for free.** The variance described by the PPD decomposes into epistemic uncertainty (reducible with more data) and aleatoric uncertainty (irreducible noise) — essential for responsible deployment in high-stakes settings.

- **Natural target for meta-learning.** The PPD is the unique prediction that optimally combines the prior learned during meta-training with the evidence in the context. Minimising its negative log-likelihood is the right training objective

From point prediction to distributions

Bayesian linear regression

Supposing that the parameter vector has a prior distribution $\beta \sim \mathcal{N}(\beta; 0, \tau^2 I)$

and that observations are generated as per $p(y_i|x_i, \beta) = \mathcal{N}(y_i; \beta^T x_i, \sigma^2)$

We have that $p(\beta|X_{\text{train}}, y_{\text{train}}) = p(\beta|\mathcal{C}) = \mathcal{N}(\beta; \mu_{\text{post}}, \Sigma_{\text{post}})$

context

with $\Sigma_{\text{post}} = \left(\frac{1}{\sigma^2} X_{\text{train}}^T X_{\text{train}} + \frac{1}{\tau^2} I\right)^{-1}$ and $\mu_{\text{post}} = \frac{1}{\sigma^2} \Sigma_{\text{post}} X_{\text{train}}^T y_{\text{train}}$

NB: Note what happens when $\tau \to \infty$

From point prediction to distributions

The PPD is obtained by marginalizing out the coefficients as in

Gaussian integral

$$p(y_\star|x_\star,\mathcal{C}) = \int p(y_\star|x_\star,\beta,\mathcal{C})p(\beta|\mathcal{C})\mathrm{d}\beta = \boxed{\int p(y_\star|x_\star,\beta)p(\beta|\mathcal{C})\mathrm{d}\beta}$$

which can be simplified to

$$p(y_\star|x_\star,\mathcal{C}) = \mathcal{N}\left(y_\star; x_\star^T\mu_{\text{post}}, x_\star^T\Sigma_{\text{post}}x_\star + \sigma^2\right)$$

Note that to compute the prediction we first **convert the context** $\mathcal{C}$ into a pair of parameters $(\mu_{\text{post}}, \Sigma_{\text{post}})$ which is then used to calculate the prediction $y_\star$

This can be seen as a primitive **kind of meta-estimator** taking datasets as input.

From point prediction to distributions

It is natural to want to consider richer features than the linear case

$$\boxed{y_\star = \beta^T x_\star} \longrightarrow \boxed{y_\star = \beta^T \phi(x_\star)} \quad \text{with} \quad \phi(x_\star) \in \mathbb{R}^m$$

which also yields a closed-form expression for PPD

$$p(y_\star | x_\star, \mathcal{C}) = \mathcal{N}\Big(y_\star; \phi(x_\star)^T \mu_{\text{post}}, \phi(x_\star)^T \Sigma_{\text{post}} \phi(x_\star) + \sigma^2\Big)$$

Interestingly, we can rewrite this expression only in terms of inner products as per

$$\mathbb{E}[y_\star | x_\star, \mathcal{C}] = K_{x_\star}^T (I\sigma^2 + K)^{-1} y_{\text{train}} \quad \text{and} \quad \text{Var}(y_\star | x_\star, \mathcal{C}) = k(x_\star, x_\star) - K_{x_\star}^T (I\sigma^2 + K)^{-1} K_{x_\star} + \sigma^2$$

with $\quad k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad K_{x_\star} \in \mathbb{R}^n \quad [K_{x_\star}]_i = k(x_i, x_\star) \quad K \in \mathbb{R}^{n \times n} \quad [K]_{i,j} = k(x_i, x_j)$

The choice of features can therefore be **turned into a choice of kernel**

- Introduction
- From point prediction to distributions
- Gaussian processes
- Conditional neural processes
- Tabular foundational models

Gaussian processes

We are interested in situations for which the data is generated as per $y = f(x) + \epsilon$

$$\left[\ x_1 \quad \dots \quad x_n\ \right] \longrightarrow \left[\ f(x_1) \quad \dots \quad f(x_n)\ \right] \longrightarrow \left[\ y_1 \quad \dots \quad y_n\ \right]$$

Note that when fixing a prior over β we're also fixing a **prior over the function** values!

$$\boldsymbol{f} = \begin{bmatrix} f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix} = \begin{bmatrix} \beta^T \phi(x_1) \\ \vdots \\ \beta^T \phi(x_1) \end{bmatrix} = \Phi\beta \quad \text{with} \quad \Phi = \begin{bmatrix} \phi(x_1) \\ \vdots \\ \phi(x_n) \end{bmatrix} \quad \text{and} \quad \beta \sim \mathcal{N}(\beta; 0, \tau^2 I)$$

So that $\boldsymbol{f} = \Phi\beta \sim \mathcal{N}(\boldsymbol{f}; 0, \tau^2 \Phi\Phi^T)$ and $[\Phi\Phi^T]_{i,j} = \phi(x_i)^T \phi(x_j) = k(x_i, x_j) = K_{ij}$

Therefore, $\boxed{\boldsymbol{f} \sim \mathcal{N}(\boldsymbol{f}; 0, K)}$

meaning that the **function values are characterized directly by the kernel**

Gaussian processes

Note that when taking the taking the dimension of the features to $m \to \infty$

the vector $\beta \in \mathbb{R}^m$ becomes **infinite-dimensional** and can no longer be written!

However, the joint pdf over the function values is still **perfectly well-defined**

$$\boldsymbol{f} = \begin{bmatrix} f(x_1) \\ \vdots \\ f(x_n) \end{bmatrix} \sim \mathcal{N}(\boldsymbol{f}; 0, K)$$

We call this object a Gaussian process and denote it as $\boldsymbol{f} \sim \mathcal{GP}(0, k)$

The function k is called the **covariance kernel**; choosing it is equivalent to choosing a prior over functions: it encodes beliefs about what the function looks like before seeing any data.

Gaussian processes

Given context $\mathcal{C} = (X_{\text{train}}, y_{\text{train}})$, the PPD for a Gaussian process write just as before:

$$\boxed{p(y_\star | x_\star, \mathcal{C}) = \mathcal{N}\Big(y_\star; K_{x_\star}^T (K + \sigma^2 I)^{-1} y_{\text{train}}, k(x_\star, x_\star) - K_{x_\star}(K + \sigma^2 I)^{-1} K_{x_\star} + \sigma^2\Big)}$$

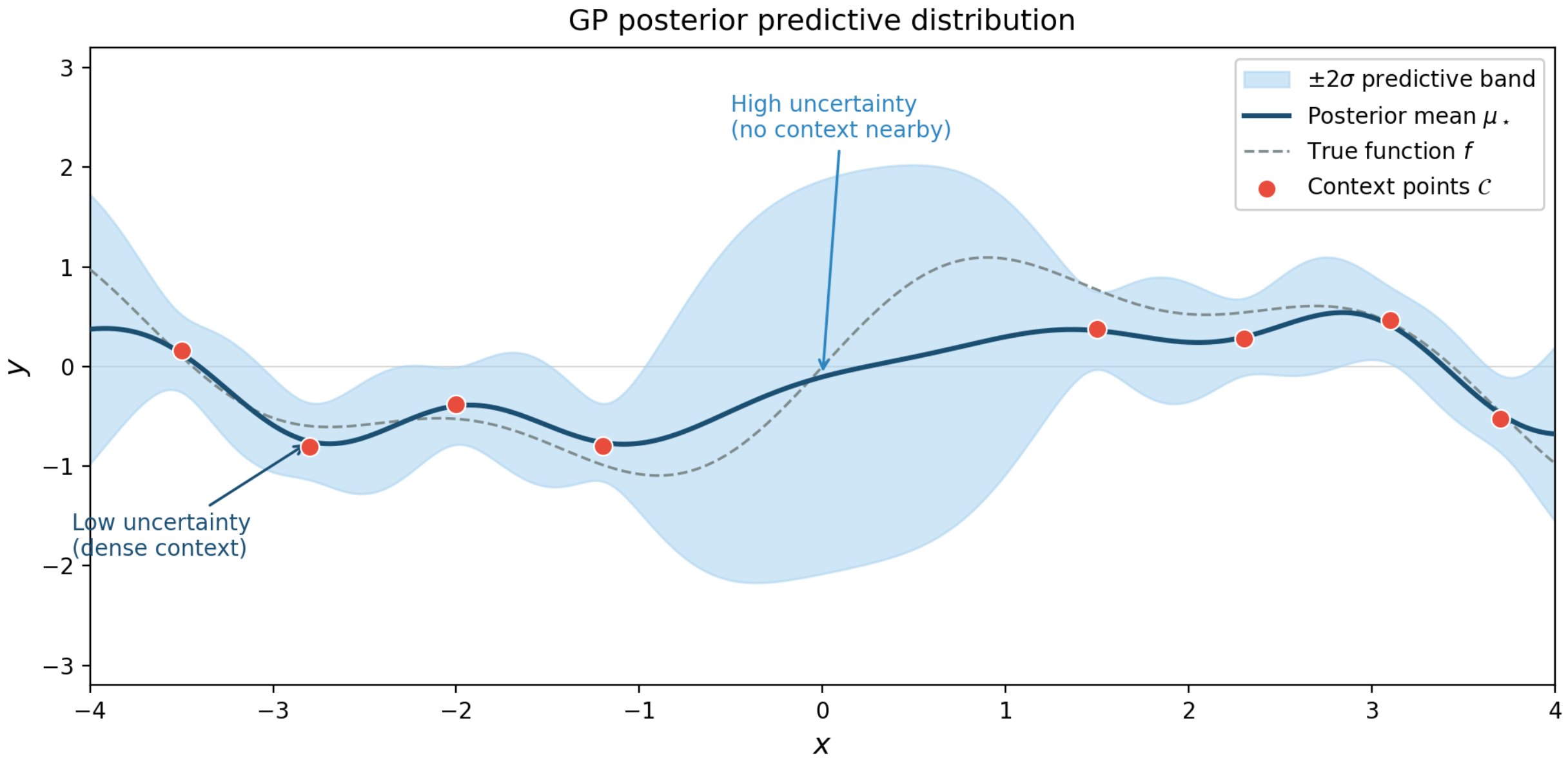
This is **exact and closed-form**. No approximation. No optimization.

> **Key insight:** the GP is the Bayesian posterior predictive that you get when the features as infinite-dimensional and the only choice you make is the kernel.

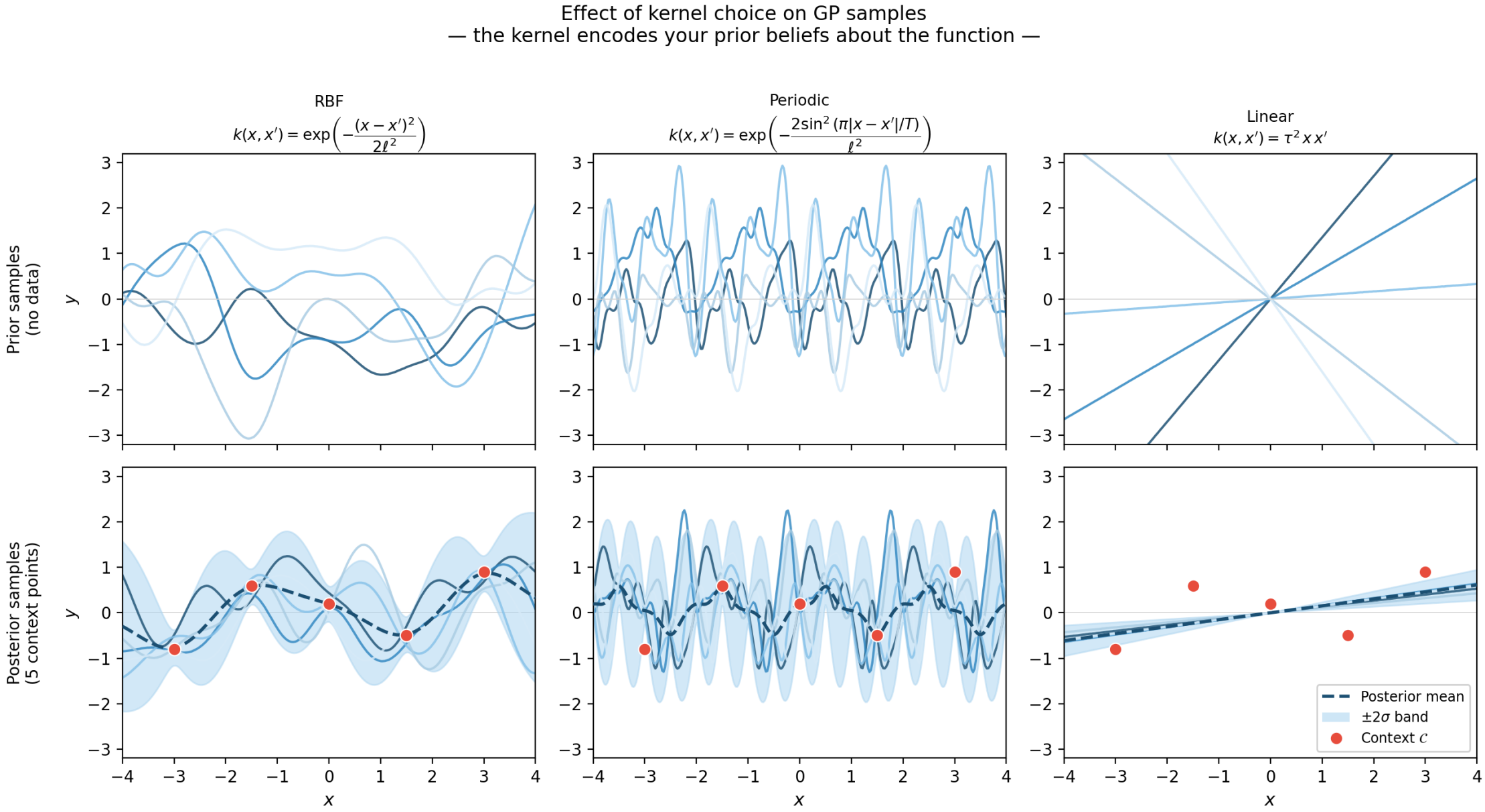
Limitations:

1. Computational cost: inverting n-by-n matrix costs $O(n^3)$ time and $O(n^2)$ memory
2. The kernel has to be (well) chosen by hand, which is not obvious at all in practice

Gaussian processes in practice – Example 1



Gaussian processes in practice – Example 2



- Introduction
- From point prediction to distributions
- Gaussian processes
- Conditional neural processes
- Tabular foundational models

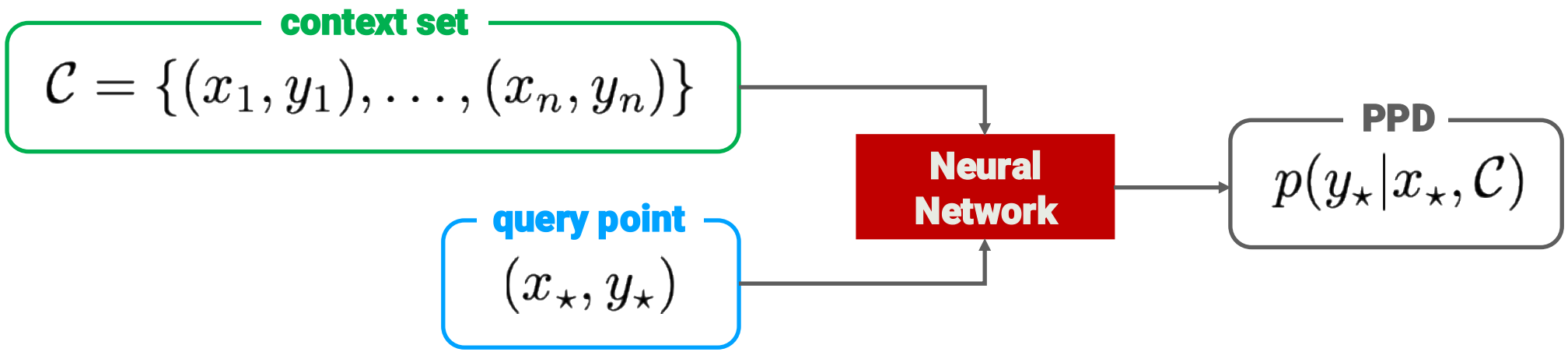
Conditional neural processes

The Gaussian process gives us the exact posterior predictive distribution (PPD), but

- at $O(n^3)$ cost and
- with a hand-chosen kernel

Q: Can we do better with a neural network?

Note that we need a neural network capable of doing the same mapping as a GP, i.e.



Conditional neural processes

Conditional neural processes (Garnelo et al. 2018) approximate the PPD with a neural network composed of three components :

embedding independent of the index

1) Encoder

A trainable function is applied to each context pair

$$(x_i, y_i) \in \mathcal{C} \longrightarrow \blacksquare \longrightarrow r_i = h_\theta(x_i, y_i) \in \mathbb{R}^d$$

preserves permutation-invariance

2) Aggregator

The individual representations are averaged into a single global

$$\begin{matrix} r_1 \\ \vdots \\ r_n \end{matrix} \longrightarrow \bullet \longrightarrow r = \frac{1}{n} \sum_{i=1}^{n} r_i$$

3) Decoder

A trainable function outputs the parameters of a predictive Gaussian for the PPD

$$x_\star \quad r \longrightarrow \blacksquare \longrightarrow (\mu_\star, \sigma_\star) = g_\phi(x_\star, r)$$

$$\boxed{p(y_\star | x_\star, \mathcal{C}) \approx \mathcal{N}(y_\star; \mu_\star, \sigma_\star^2)}$$

Conditional neural processes

Q: OK, but how do we train a CNP?

Note that we have two parameters to estimate : θ and ϕ

Related to the encoding

Related to the decoding

We need a loss function depending on both parameters as per $\mathcal{L}(\theta, \phi)$

so that we can use gradient descent: $$\begin{bmatrix} \theta_{k+1} \\ \phi_{k+1} \end{bmatrix} = \begin{bmatrix} \theta_k \\ \phi_k \end{bmatrix} - \eta \nabla \mathcal{L}(\theta_k, \phi_k)$$

New question: which loss function to use then?

Conditional neural processes

New question: which loss function to use?

Suppose you have a model that outputs not just a prediction $\hat{y}_i$ but a full Gaussian distribution $\mathcal{N}(y_i; \hat{\mu}_i, \hat{\sigma}_i^2)$ for the values of y_i

How do you measure whether this predicted distribution is good or bad?

Conditional neural processes

New question: which loss function to use?

Suppose you have a model that outputs not just a prediction $\hat{y}_i$ but a full Gaussian distribution $\mathcal{N}(y_i; \hat{\mu}_i, \hat{\sigma}_i^2)$ for the values of y_i

How do you measure whether this predicted distribution is good or bad?

With a point prediction this would be easy – compute the squared error $(y_i - \hat{y}_i)^2$

For a whole distribution, we ask: how much probability did your distribution assign to the value that **actually happened**?

$$p(y_i; \hat{\mu}_i, \hat{\sigma}_i^2) = \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(y_i - \hat{\mu}_i)^2}{2\hat{\sigma}_i^2}\right)$$

↑ good model for that data point

↓ bad model for that data point

Conditional neural processes

New question: which loss function to use?

For a dataset containing several IID query points $\mathcal{T} = \left\{(x_i, y_i)\right\}_{i=1}^{N_\mathcal{T}}$

we can write the loss function as

$$\ell(\theta, \phi) = \prod_{i=1}^{N_\mathcal{T}} p(y_i; \hat{\mu}_i, \hat{\sigma}_i^2) \quad \Leftrightarrow \quad \mathcal{L}(\theta, \phi) = -\sum_{i=1}^{N_\mathcal{T}} \log p(y_i; \hat{\mu}_i, \hat{\sigma}_i^2)$$

(Note that the parameters being opimized appear impliclity in the calcuation of the means and variances)

and specifically for the Gaussian model case we get

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^{N_\mathcal{T}} \left[\frac{1}{2}\log(2\pi\hat{\sigma}_i^2) + \frac{(y_i - \hat{\mu}_i)^2}{2\hat{\sigma}_i^2}\right]$$

Conditional neural processes

Q: OK, but how do we train a CNP?

We use a **meta-learning** procedure

run several episodes like this

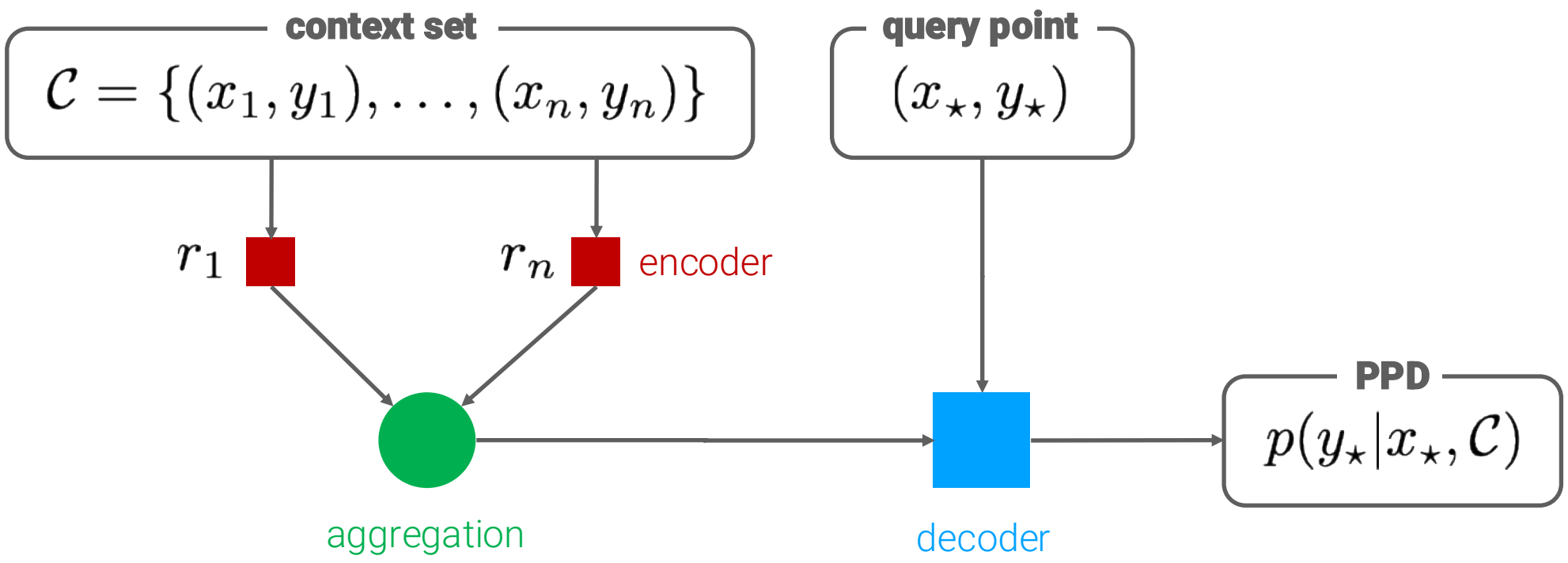
1. Sample a function from a meta-training distribution, e.g. $f \sim \mathcal{GP}(0, k)$
2. Sample N input locations and evaluate $y_i = f(x_i) + \epsilon_i$
3. Randomly split into context set $\mathcal{C}$ and target set $\mathcal{T}$
4. Run the CNP forward pass on the context to obtain a (μ_j, σ_j) for each $x_j \in \mathcal{T}$
5. Compute the negative log-likelihood loss

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^{N_{\mathcal{T}}} \left[\frac{1}{2} \log(2\pi\hat{\sigma}_i^2) + \frac{(y_i - \hat{\mu}_i)^2}{2\hat{\sigma}_i^2} \right]$$

6. Calculate the gradient of the loss and do a gradient step

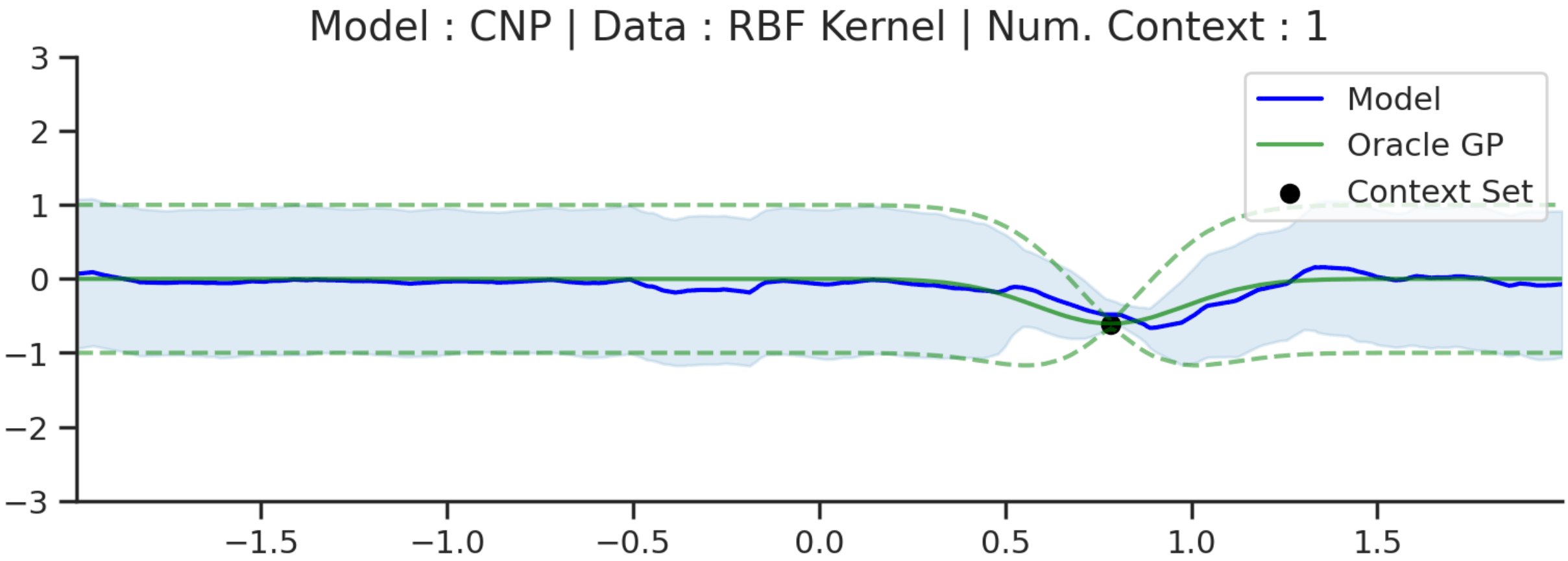
Conditional neural processes

After millions (!) of steps in the meta-learning procedure, the network will have learned a general strategy for predicting functions from partial observations in a context.



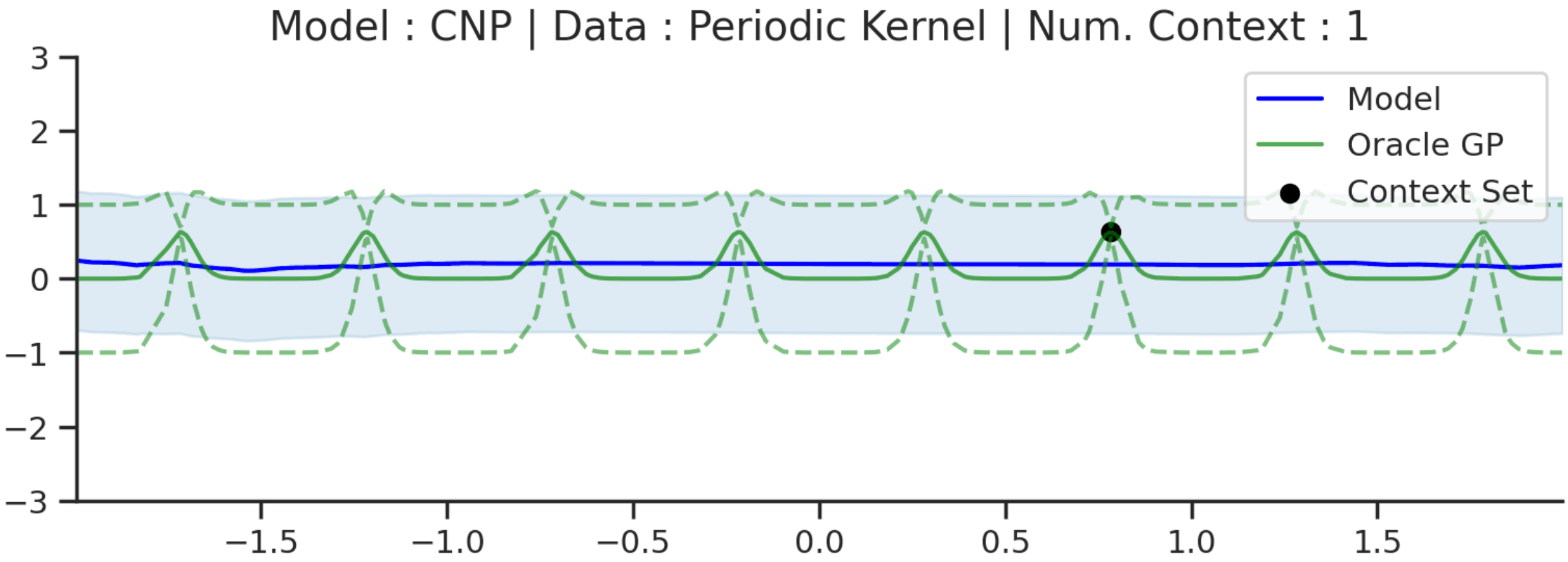
Compared to a GP, we have a O(n) complexity and a supervised data-driven kernel

Conditional neural processes



Reference: https://yanndubs.github.io/Neural-Process-Family/text/CNPF.html

Conditional neural processes



Reference: https://yanndubs.github.io/Neural-Process-Family/text/CNPF.html

- Introduction
- From point prediction to distributions
- Gaussian processes
- (Conditional) neural processes
- Tabular foundational models

Tabular foundational models

Despite their elegance, conditional neural processes have important weaknesses

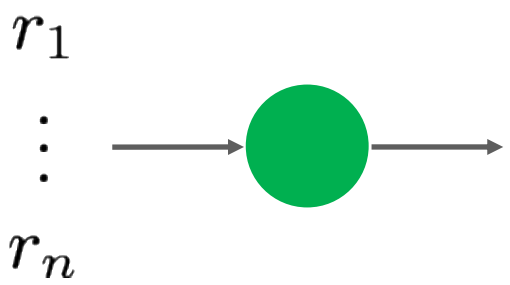
Tabular foundational models

Despite their elegance, conditional neural processes have important weaknesses

1. **Lossy aggregator:** the mean pooling throws away all information about which y came from which x. If two context points have very different y values at nearby x locations, the mean r cannot represent this. The model underfits local structure.

Aggregator

The individual representations are averaged into a single global



$$r = \frac{1}{n}\sum_{i=1}^{n} r_i$$

preserves permutation-invariance

Tabular foundational models

Despite their elegance, conditional neural processes have important weaknesses

1. **Lossy aggregator:** the mean pooling throws away all information about which y came from which x. If two context points have very different y values at nearby x locations, the mean r cannot represent this. The model underfits local structure.
2. **The prior over functions is not sufficiently rich:** the CNPs are good at smooth regressions functions but know nothing about tabular classification and regression problems with heterogeneous features, mixed scales, class imbalance, etc.

recall the first two steps of meta-learning

1. Sample a function from a meta-training distribution, e.g. $f \sim \mathcal{GP}(0, k)$
2. Sample N input locations and evaluate $y_i = f(x_i) + \epsilon_i$

Tabular foundational models

Despite their elegance, conditional neural processes have important weaknesses

1. **Lossy aggregator:** the mean pooling throws away all information about which y came from which x. If two context points have very different y values at nearby x locations, the mean r cannot represent this. The model underfits local structure.
2. **The prior over functions is not sufficiently rich:** the CNPs are good at smooth regressions functions but know nothing about tabular classification and regression problems with heterogeneous features, mixed scales, class imbalance, etc.
3. **A Gaussian model is hardly enough:** the Gaussian output for a CNP is not enough for regression problems with multimodal, skewed, or heavy-tailed noise. Also, we simply can't apply it to classification where the y are discrete labels.

Tabular foundational models

- **Fix 1 : Use an attention mechanism for context**

In CNP, a representation is the same for every target query.

For instance, when predicting $y_\star$ for a given $x_\star$ that is near to context point (x_3, y_3) the model cannot give extra weight to (x_3, y_3) since all points contribute equally.

Tabular foundational models

- **Fix 1 : Use an attention mechanism for context**

In CNP, a representation is the same for every target query.

For instance, when predicting $y_\star$ for a given $x_\star$ that is near to context point (x_3, y_3) the model cannot give extra weight to (x_3, y_3) since all points contribute equally.

We can fix this by using a a context representation that is **query-specific**.

$$r(x_\star) = \sum_{i=1}^{n} \alpha_i(x_\star) v_i \quad \text{with} \quad \alpha_i(x_\star) = \frac{\exp(q_\star^T k_i / \sqrt{d})}{\sum_{j=1}^{n} \exp(q_\star^T k_j) / \sqrt{d}}$$

$$k_i = K h_\theta(x_i, y_i)$$

keys are learned projections of the context representations

$$v_i = V h_\theta(x_i, y_i)$$

values are learned projections of the context representations

$$q_\star = Q x_\star$$

the query is a learned projection of the target input

Tabular foundational models

- **Fix 1 : Use an attention mechanism for context**

In CNP, a representation is the same for every target query.

For instance, when predicting $y_\star$ for a given $x_\star$ that is near to context point (x_3, y_3) the model cannot give extra weight to (x_3, y_3) since all points contribute equally.

We can fix this by using a a context representation that is **query-specific**.

$$r(x_\star) = \sum_{i=1}^{n} \alpha_i(x_\star) v_i \quad \text{with} \quad \alpha_i(x_\star) = \frac{\exp(q_\star^T k_i / \sqrt{d})}{\sum_{j=1}^{n} \exp(q_\star^T k_j) / \sqrt{d}}$$

This is **cross-attention**: the query comes from target, the keys+values from context

↳ We get an embedding $r(x_\star)$ that is constructed **focusing on the parts of the context** that are most relevant for predicting that location

Tabular foundational models

Parenthesis: some good places to read and learn about the attention mechanism

Original paper by Google researchers

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

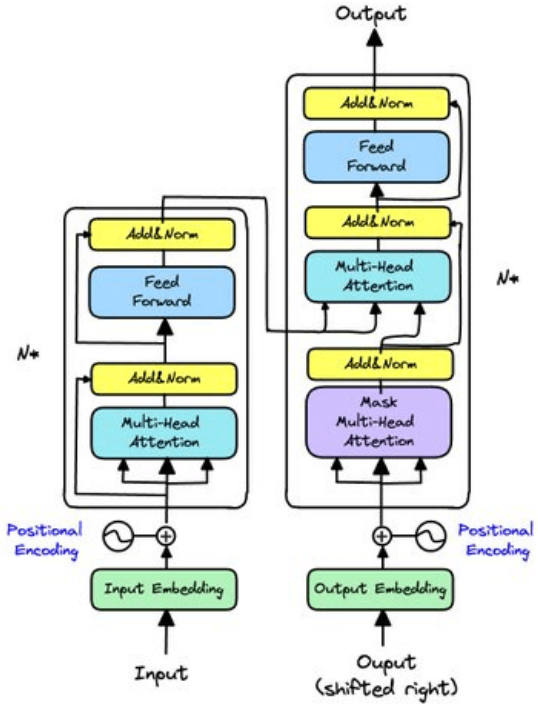
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

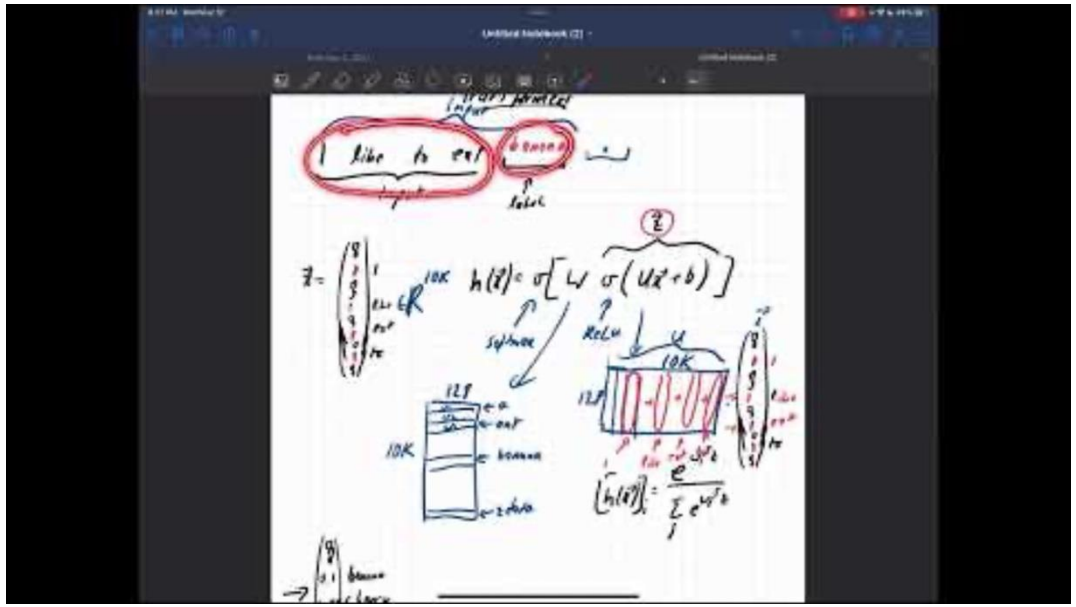
*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.



Excellent video on attention and transformers



https://www.youtube.com/watch?v=Nu7VZeW5TaE

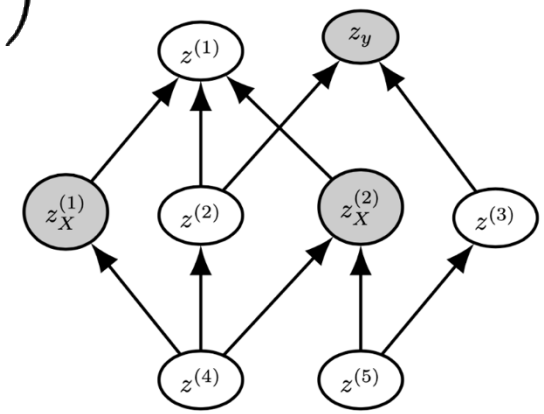
Tabular foundational models

- **Fix 2 : Leverage synthetic data to get a better prior**

We want a prior that covers the diversity of real tabular datasets: heterogeneous features, nonlinear dependencies, mixed scales, class imbalance, etc.

💡 Sample datasets from a structured causal model (SCM), which consists of

1. A set of variables $X_1, \ldots, X_n, Y$
2. A directed acyclic graph (DAG) specifying which variables influence which
3. A function form for each variable $X_j = f_j\Big(\text{parents}(X_j, \varepsilon_j)\Big)$



Tabular foundational models

- **Fix 2 : Leverage synthetic data to get a better prior**

To sample one synthetic dataset we:

1. Sample a DAG structure (number of nodes, edge probabilities, etc)
2. Sample random functional form f_j (linear, polynomial, etc.)
3. Sample random noise distribution
4. Sample n realisations from this SCM to get a dataset (X, y)

Each sampled SCM defines a different task. The distribution over SCMs defines a prior over tasks. By varying the DAG structure, the functional forms, the noise distributions, number of features, etc., we cover a **very broad class of tabular tasks**

Tabular foundational models

- **Fix 3 : Richer output distribution**

The decoder of a CNP (i.e. its output) is a Gaussian distribution approximating the PPD

$$p(y_\star | x_\star, \mathcal{C}) \approx \mathcal{N}(y_\star; \mu_\star, \sigma_\star^2)$$

However, if the target variables are discrete in a classification task it won't work…

Tabular foundational models

- **Fix 3 : Richer output distribution**

The decoder of a CNP (i.e. its output) is a Gaussian distribution approximating the PPD

$$p(y_\star | x_\star, \mathcal{C}) \approx \mathcal{N}(y_\star; \mu_\star, \sigma_\star^2)$$

However, if the target variables are discrete in a classification task it won't work...

What we do is to consider a C-dimensional vector of **logits** $z \in \mathbb{R}^C$ defined as

$$z = W_{\text{out}} \underbrace{r(x_\star)}_{\text{output of the cross-attention layer}} + b_{\text{out}} = \begin{bmatrix} z_1 \\ \vdots \\ z_C \end{bmatrix} \quad \text{and} \quad p(y_\star = c | x_\star, \mathcal{C}) = \underbrace{\frac{\exp(z_c)}{\sum_{i=1}^{C} \exp(z_i)}}_{\text{softmax operation}}$$

(note that W_{out} and b_{out} are both learnable parameters!)

Tabular foundational models

Fix 3 : Richer output distribution

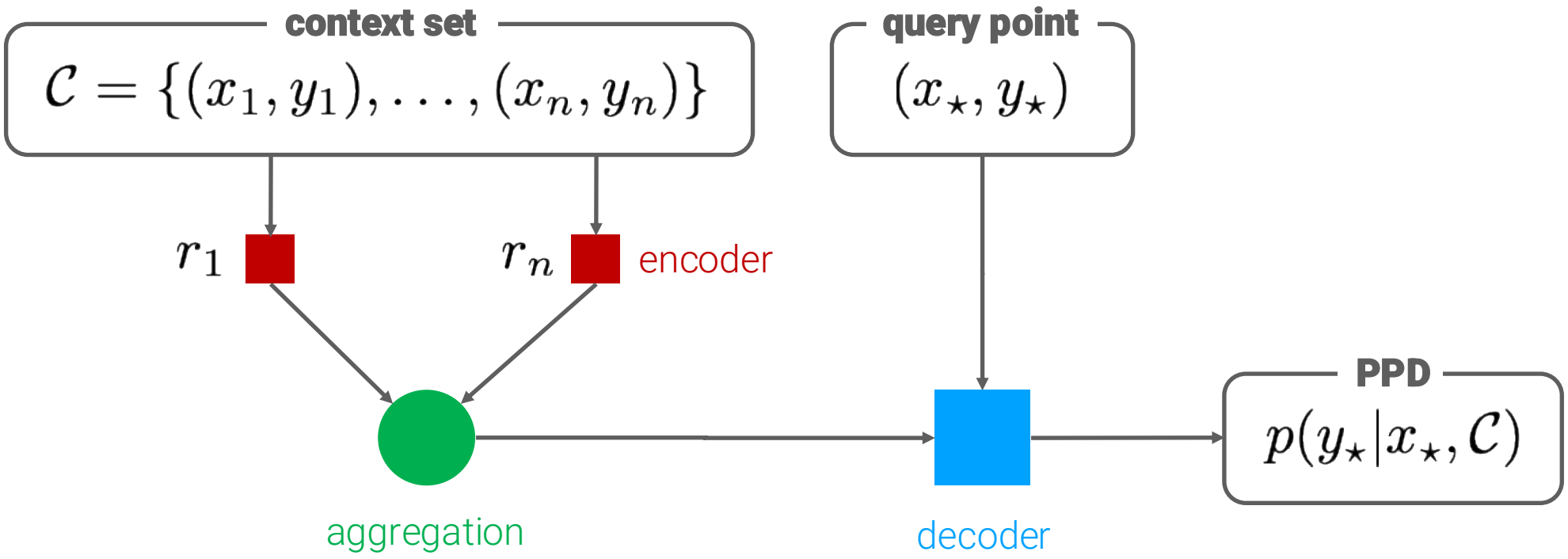
The new loss function for classification tasks becomes the cross-entropy loss

$$\mathcal{L} = -\sum_{i=1}^{N_\tau} \log p(y_i | x_i, \mathcal{C}) = \sum_{i=1}^{N_\tau} \left[-z_{y_i} + \log \sum_{c=1}^{C} \exp(z_c) \right]$$

	Gaussian decoder	Softmax decoder
Output	(μ_i, σ_i^2)	$(z_1, \ldots, z_C)$
Predicted distribution	$\mathcal{N}(y_i; \mu_i, \sigma_i^2)$	$\exp(z_{y_i}) / \sum_c \exp(z_c)$
Loss per point	$\frac{1}{2}\log(2\pi\sigma_i^2) + \frac{(y_i-\mu_i)^2}{2\sigma_i^2}$	$-z_{y_i} + \log \sum_c \exp(z_c)$
Penalty for uncertainty	$\frac{1}{2}\log \sigma_i^2$ grows if σ large	$\log \sum_c \exp(z_c)$ grows if logits spread
Penalty for error	$(y_i - \mu_i)^2 / 2\sigma_i^2$	$-z_{y_i}$ pushes correct logit up

Tabular foundational models

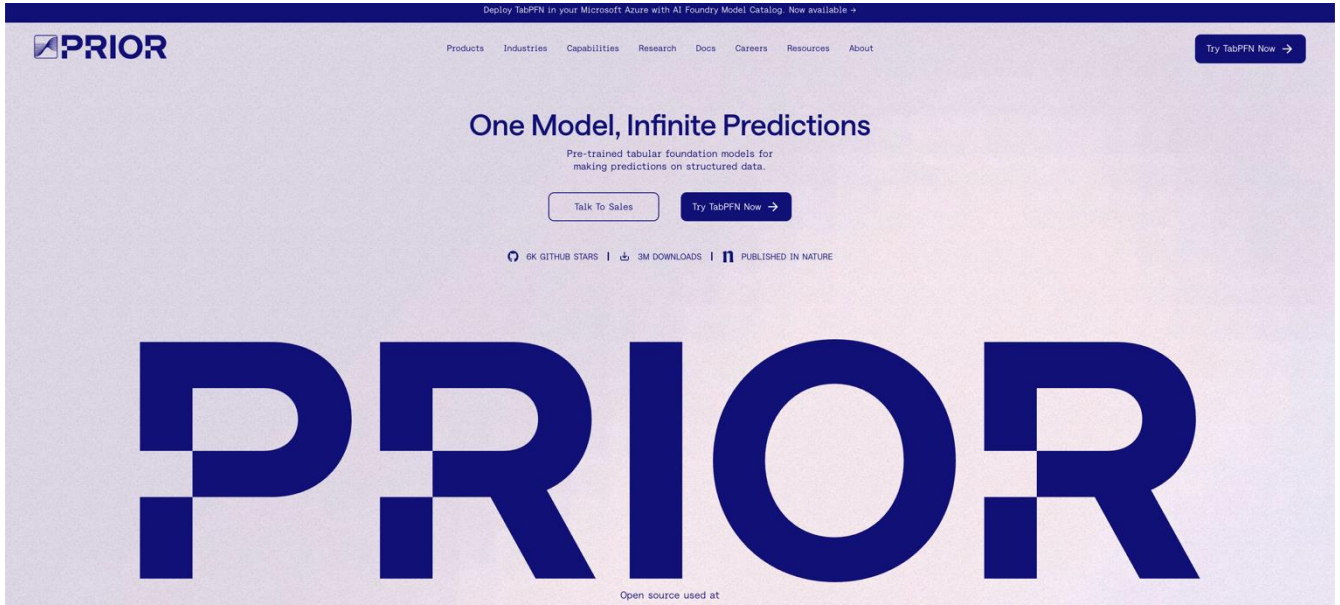
Putting it all together, we get a tabular foundational model

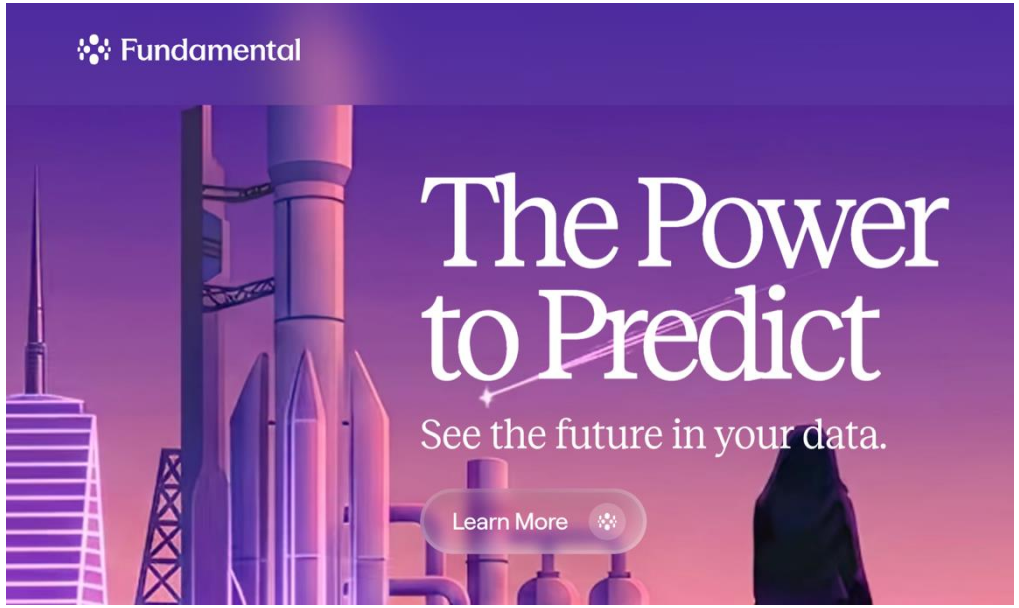


The two most popular architectures are TabPFN and TabICL, which are somewhat more complicated, but fundamentally what we have seen.

Tabular foundational models

There's a lot of $$ going around this area…





Which is why many ideas and implementations are not open-source

TLDR: tabular foundational models are changing data science jobs profoundly